Moiré-Effekte in der Visualisierung

Ullika Christiane Scholz
Freie Universität Berlin
Institut für Mathematik

Bachelorarbeit
Betreuer: Prof. Dr. Konrad Polthier

6. Oktober 2014

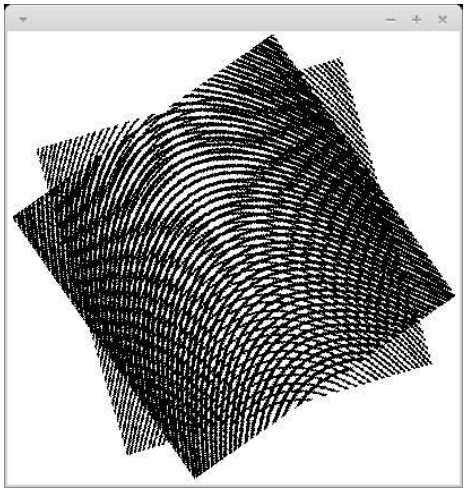


Abbildung 1: Mit MOIGEN generiertes Moiré

Ziel dieser Bachelorarbeit ist es, ein mathematisches Modell vorzustellen, durch das sich Moiré-Effekte mithilfe der Fourier-Theorie vorhersagen und klassifizieren lassen. Dabei soll der Leser einen guten Überblick über verschiedene Arten von Moirés bekommen und die Verbindung zur Abtastproblematik erkennen.

Die Struktur werde ich dafür weitgehend vom ersten Kapitel des Buches [Ami09] übernehmen, das auf Isaac Amidrors eigenem Artikel [Ami94] beruht. Die Grundlagen der Fourier-Theorie entnehme ich [Bra78] von Ronald N. Bracewell und [Kö88] von T.W. Körner.

Um den Leser selbst experimentieren zu lassen, überlasse ich ihm mein selbst geschriebenes Programm MOIGEN. Damit hat er die Möglichkeit, Gitter und Raster verschiedener Art übereinander zu legen und die theoretischen Ergebnisse praktisch zu überprüfen.

Inhaltsverzeichnis

1 Einleitung **3**

2 Fourier-Theorie und Signalverarbeitung **6**
2.1 Fourier-Reihen und die Fourier-Transformation 6
2.2 Die Konvolution 7
2.3 Abtastung 7

3 Der spektrale Ansatz **8**
3.1 Binäre Liniengitter 10
3.2 Gitternetze 13
3.3 Punktraster 14
3.4 Sampling-Moirés 15
3.5 Weitere Strukturen 17

4 Das Programm MOIGEN **19**

1 Einleitung

Moiré-Effekte entstehen, wenn zwei oder mehr sich wiederholende Strukturen gegeneinander betrachtet werden. Dabei werden Muster sichtbar, die in keiner der ursprünglichen Strukturen vorhanden waren. Hat man erst einen Blick dafür entwickelt, begegnet man Moiré-Effekten ständig, zum Beispiel an Treppengeländern, Zäunen oder gewebten Stoffen. Moiré-Stoffe sind schon Jahrhunderte unter diesem Namen bekannt und mittlerweile bezeichnet das Wort Moiré, das zunächst einer bestimmten Textilie vorbehalten war, ein ganzes Phänomen. Meist tritt der Effekt ungewollt auf, manchmal wird er aber auch zu speziell künstlerischen Zwecken eingesetzt, wie im Naturkundemuseum Berlin, wo die schimmernden Quadrate an Insektenaugen erinnern, oder in den Arbeiten des niederländischen Künstlers Koert Feenstra.



Abbildung 2: Stellwände im Naturkundemuseum Berlin



Abbildung 3: Ein Brückengeländer in Berlin-Kreuzberg

Der Effekt entsteht durch eine Interaktion der übereinandergelegten Schichten. Wie die Abb. 6 zeigt, ist ein Bild entlang der Bereiche, in denen sich die dunklen Stellen der ursprünglichen Strukturen überlagern, heller als dort, wo die dunklen Stellen nebeneinander auftreten, mehr Platz einnehmen und deshalb weniger Licht reflektieren oder durchlassen. Dies erkannte schon der Physiker Lord Rayleigh, der im Jahr 1874 in [Ray74] folgende Bemerkung zum Moiré-Effekt machte:



Abbildung 4: Eine Obstschale im Schaufenster



Abbildung 5: Ein Kleidungsstück

> “If two photographic copies containing the same number of lines to the inch be placed in contact, film to film, in such a manner that the lines are nearly parallel in the two gratmgs, a system of parallel bars develops itself, whose direction bisects the external angle between the directions of the original lines, and whose distance increases as the angle of inclination diminishes. The cause of the phenomenon will be readily understood by drawing on paper two sets of equally distant and not too thin bars inclined at a small angle. Where the opaque and tranparent parts severally overlap, the obstruction of light is, on the average, less than the double of that due to each set separately, and consequently these places appear by comparison bright.”

Rayleigh war einer der ersten Wissenschaftler, der sich darum bemühte, eine Theorie des Moiré-Phänomens aufzustellen. Etwa hundert Jahre lang folgte man dabei einem geometrischen Ansatz, der die geometrischen Eigenschaften der ursprünglichen Strukturen untersuchte und davon auf resultierende Moiré-Muster schloss. Noch 1967 erschienen Artikel wie [AJD67] von den US-amerikanischen Wissenschaftlern A. J. Durelli und V. J. Parks, in denen Parametergleichungen für die beteiligten Linien und die resultierenden Moiré-Muster aufgestellt werden.

Um 1970 gelang ein Durchbruch mit der Verwendung der Fourier-Theorie, die eine Analyse der Ursprungsbilder sowie der Überlagerung auf spektraler Ebene

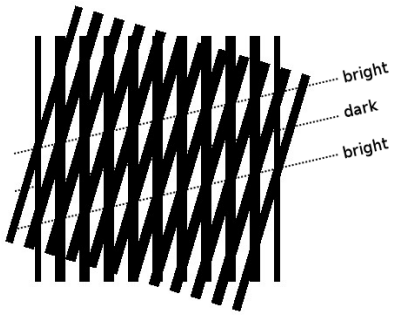


Abbildung 6: Entstehung von Moiré nach [Ami09]

möglich machte. Dafür war die Arbeit [Bry74] des Physikers Olof Bryngdahl von großer Bedeutung, welcher als Vater der in dieser Arbeit vorgestellten Theorie gelten kann. Ebenfalls über die Fourier-Theorie gelang es, die Entstehung von Moiré mit dem Problem der Abtastung aus der Signalverarbeitung zusammenzubringen. Isaac Amidrors wissenschaftliche Leistungen sind neben dem Zusammentragen einer Vielzahl von verstreuten Informationen eine genaue Untersuchung der Überlagerung von Punktrastern und eine Theorie zur Intensität der resultierenden Moiré-Muster.

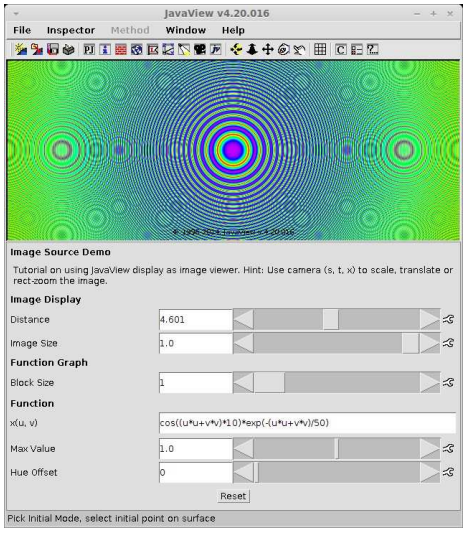


Abbildung 7: Moiré im Visualisierungsprogramm JavaView



Abbildung 8: Moiré auf einem digital aufgenommenen Foto (Quelle: [Pel12])

2 Fourier-Theorie und Signalverarbeitung

Bevor dem Leser die Theorie des Moiré-Phänomens vorgestellt wird, komme ich nicht umhin, einige Grundbegriffe und Erkenntnisse aus Fourier-Theorie und Signalverarbeitung zu erklären.

2.1 Fourier-Reihen und die Fourier-Transformation

Der französische Mathematiker und Physiker Joseph Fourier zeigte um 1820, dass es möglich ist, allgemeine periodische Funktionen als unendliche Reihe von Sinus- und Kosinus-Funktionen aufzufassen. Vorher waren nur Reihenentwicklungen für bestimmte Funktionen bekannt. Wenig später stellte er die Fouriertransformation als Verallgemeinerung der Reihenentwicklung vor. Angewendet auf aperiodische Funktionen, vermag sie auch diese in ihre überabzählbar vielen periodischen Komponenten zu zerlegen. Die Fouriertransformierte einer Funktion $f(x, y)$ ist eine zweite, neue Funktion $F(u, v)$, deren Funktionswerte an die Stelle der abzählbaren Menge von Koeffizienten c_{mn} der Fourier-Reihenentwicklung treten. Da $F(u, v)$ die Zusammensetzung von $f(x, y)$ aus Sinus- und Kosinusfunktionen beschreibt, heißt sie das *Frequenzspektrum* von f.

Obwohl Fouriers Ideen speziell für die Lösung der Wärmeleitungsgleichung vorgesehen waren, stellten sich die Fourier-Reihenentwicklung und die Fourier-Transformation schnell als unentbehrliche Werkzeuge in Mathematik und Physik heraus.

Satz 1. *Für eine reellwertige, entlang der Vektoren $P_1 = (x_1, y_1)$ und $P_2 = (x_2, y_2)$ periodische Funktion $p(x, y)$ in der Ebene existiert eine Reihenentwicklung*

$$p(x, y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} c_{mn} e^{i2\pi[(mu_1+nu_2)x+(mv_1+nv_2)y]}$$

mit den Frequenzvektoren

$$f_1 = (u_1, v_1) = \frac{1}{A}(y_2 - x_2)$$

und

$$f_2 = (u_2, v_2) = \frac{1}{\overline{A}}(-y_1, x_1)$$

und den Koeffizienten

$$c_{mn} = \frac{1}{\overline{A}} \iint\limits_A p(x,y) e^{-i2\pi(mu_0 x + nv_0 y)} dxdy,$$

wobei A das von P_1 und P_2 aufgespannte Parallelogramm mit Flächeninhalt

$$\overline{A} = P_1 \times P_2 = x_1 y_2 - y_1 x_2$$

ist.
Satz 2. *Die Fourier-Transformierte $F(u,v)$ ist für eine Riemann-integrierbare Funktion $f(x,y)$ wohldefiniert und wird berechnet durch*

$$\mathcal{F}[f] = F(u,v) = \int\limits_{-\infty}^{\infty} \int\limits_{-\infty}^{\infty} f(x,y) e^{-i2\pi(ux+vy)} dxdy.$$

2.2 Die Konvolution

Die Konvolution oder Faltung ist ein mathematischer Operator, der aus den Funktionen f und g eine dritte Funktion $f * g$ generiert. Ist der Beobachter daran interessiert, welchen Wert $f * g$ an der Stelle (x_1, y_1) annimmt, muss er sich vorstellen, dass der an der y-Achse gespiegelte Funktionsgraph von f um den Vektor (x_1, y_1) unter den Funktionsgraphen von g geschoben wird. Das Ergebnis $(f * g)(x_1, y_1)$ entspricht dem Integral des Produkts der übereinander liegenden Funktionswerte über die gesamte Ebene. Diese Methode stellt Bracewell in [Bra78] als "Move-And-Multiply"-Methode vor.
Definition 3. *Für $f, g : \mathbb{R}^2 \to \mathbb{R}$ gilt*

$$(f * g)(x_1, y_1) = \iint_{\mathbb{R}^2} f(x_1 - x_2, y_1 - y_2)) g(x_2, y_2) dx_2 dy_2.$$

2.3 Abtastung

Unter Abtastung (englisch: Sampling) versteht man die Umwandlung eines kontinuierlichen Signals in ein diskretes Signal. Dafür wird das kontinuierliche Signal an endlich vielen Punkten gemessen.

Unter dem Umstand, dass die Abtastrate hoch genug ist, ist das kontinuierliche Signal dabei vollständig aus dem diskreten Signal rekonstruierbar. Dafür muss die Abtastrate laut dem *Sampling Theorem* aus [Sha49] mindestens dem Doppelten der Maximalfrequenz des kontinuierlichen Signals entsprechen. Falls dies nicht eingehalten werden kann, so treten manchmal, keineswegs immer, eigenartige Störfrequenzen im diskreten Signal auf.

Um diesen Sachverhalt später untersuchen zu können, will ich dem Leser ein mathematisches Modell für die Abtastung einer kontinuierlichen Funktion f

vorstellen. Eine Kenntnis des Dirac-Impulses $\delta(x, y)$, der ein Grenzfall einer Folge von immer schmaler und immer höher werdenden Impulsen ist, setze ich voraus.

Das Produkt von f mit einem einzelnen Dirac-Impuls $\delta(x, y)$ an der Stelle (x_0, y_0) mit Amplitude 1 ist ein neuer Dirac-Impuls an der Stelle (x_0, y_0), dessen Amplitude dem Funktionswert $f(x_0, y_0)$ entspricht. Dies lässt sich im Abschnitt *The sifting property* in Kapitel 5 von [Bra78] nachvollziehen. Das Produkt von f und $\delta(x, y)$ ist ein diskrete Signal, die Multiplikation kann also als Abtastung an nur einem Messpunkt aufgefasst werden.

Für die Modellierung der Abtastung an mehreren Messpunkten ersetze man den Impuls $\delta(x, y)$ durch ein ganzes Bett aus Einheitsimpulsen. Das Produkt mit f ist ein Impulsbett, dessen Impulse als Amplitude die Funktionswerte von f an jeweiliger Stelle haben.

3 Der spektrale Ansatz

In diesem Abschnitt wird der Leser ein Modell für die Überlagerung von Bildern kennenlernen.

Um ein Bild mathematisch zu beschreiben, wird es mit einer Funktion in der Ebene, der Reflexionsfunktion $r(x, y)$, identifiziert. Diese ordnet jedem Punkt (x, y) auf dem Bild einen Wert zwischen 0 und 1 entsprechend seiner Lichtreflexion zu. Weil in dieser Arbeit keine farbigen Bilder behandelt werden, eignet sich diese Zuordnung für eine vollständige Beschreibung des Bildes.

Es entspricht der Wert 0 der Farbe schwarz (keine Lichtreflexion) und der Wert 1 der Farbe weiß (volle Reflexion). Werte dazwischen beschreiben verschiedene Schattierungen. Für den Fall, dass sich die Lichtquelle nicht auf der Seite des Betrachters, sondern hinter den überlagerten Strukturen befindet, handelt es sich bei $r(x, y)$ nicht um eine Reflexions- sondern um eine Transmissionsfunktion.

Die Überlagerung zweier Bilder kann als Multiplikation der Reflexionsfunktionen interpretiert werden. Zwei schwarze Bereiche übereinander ergeben wieder einen schwarzen Bereich, zwei weiße einen weißen, ein schwarzer und ein weißer einen schwarzen und so weiter. Der Funktionswert der Reflexionsfunktion des resultierenden Bildes ist das Produkt der Funktionswerte der einzelnen Reflexionsfunktionen.

$$r(x, y) = r_1(x, y) \cdot \ldots \cdot r_m(x, y)$$

Bei Moiré-Mustern handelt es sich um neue periodische Strukturen, die in der Überlagerung, nicht jedoch in den Ursprungsbildern vorhanden sind. Um sie zu untersuchen, ist es hilfreich, die Frequenzspektren der Ursprungsbilder und der Überlagerung zu betrachten.

Im linken Teil von Abb. 9 sieht man ein Bild, genauer gesagt ein Punktraster, im rechten Teil die Fouriertranformierte seiner Reflexionsfunktion, sein Frequenzspektrum. Achtung: Der Kreis um den Ursprung ist nicht Teil des Frequenzspektrums, sondern der sogenannte *visibility circle*, den der Leser weiter unten kennenlernen wird.

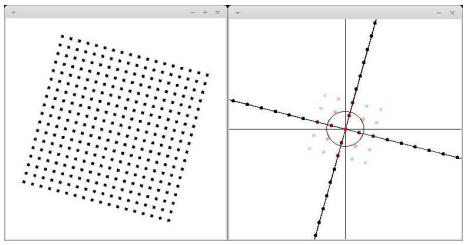


Abbildung 9: Ein Bild, erzeugt mit MOIGEN, und das zugehörige Frequenzspektrum

Nach dem Konvolutionstheorem, das zum Beispiel von Bracewell im sechsten Kapitel von [Bra78] behandelt wird, und welches besagt, dass die Multiplikation von Funktionen einer Faltung ihrer Spektren und die Faltung von Funktionen der Multiplikation ihrer Spektren entspricht, ergibt sich das Spektrum der Überlagerung durch die Faltung der Spektren der Ursprungsbilder $R_1(u,v), \dots R_m(u,v) = \mathcal{F}[r_1(x,y)] \dots \mathcal{F}[r_m(x,y)]$. Es gilt

$$R(u,v) = R_1(u,v) * \dots * R_m(u,v).$$

Weil ich in dieser Arbeit nur periodische Bilder untersuche, sind alle Reflexionsfunktionen $r_1(x,y) \dots r_m(x,y)$ periodisch. Deshalb handelt es sich bei ihren Spektren nach Appendix A in [Ami09] um Impulsbetten, deren Impulse den Koeffizienten in der Fourier-Reihenentwicklung entsprechen. Mithilfe der "Move-and-Multiply"-Methode lässt sich schließen, dass auch ihre Konvolution nicht glatt sein kann, sondern es sich bei $R(u,v)$ ebenfalls um ein Impulsbett handeln muss, dessen Impulse die Koeffizienten der Reihenentwicklung von $r(x,y)$ ausmachen. Jeder dieser Impulse im Spektrum hat drei charakteristische Eigenschaften: Sein Index in der Reihenentwicklung, seine geometrische Lage in der (u,v)-Ebene, gegeben durch den Frequenzvektor f, und seine Amplitude.

Ich werde hier ausschließlich Bilder $r_1(x,y) \dots r_m(x,y)$ behandeln, die symmetrisch an einer Spiegelachse durch den Ursprung sind. Diese Symmetrie wird an die Bildüberlagerung $r_(x,y)$ sowie nach dem Abschnitt *Significance of oddness and evenness*aus Kapitel 2 in [Bra78] an die Frequenzspektren $R_1(u,v) \dots R_m(u,v)$ und $R(u,v)$ vererbt. Das hat zur Folge, dass bis auf den Ursprungsimpuls jedem Impuls im Spektrum ein zweiter gegenüberliegt, der die gleiche Amplitude und den gespiegelten Frequenzvektor $-f$ hat.

Auf den ersten Blick mag es manchem Leser merkwürdig vorkommen, dass im Frequenzspektrum des überlagerten Bildes Impulse vorhanden sind, die nicht direkt aus einem der Ursprungsbilder stammen. Das überlagerte Bild erscheint ihm nämlich als bloße Zusammensetzung von Einzelbildern, die nicht miteinander in Interaktion treten. Wenn er sich allerdings bewusst macht, dass die Überlagerung eben keine Addition, die tatsächlich nur eine Addition der Spektren bedeuten würde, sondern eine Multiplikation ist, sollten seine Zweifel aufgehoben sein.

Ob ein Impuls im Spektrum eines Bildes ein sichtbares Moiré-Muster repräsentiert, hängt von der Sehfähigkeit des Betrachters ab. Das menschliche Auge kann periodische Strukturen nur innerhalb eines gewissen Frequenzbereiches erkennen.

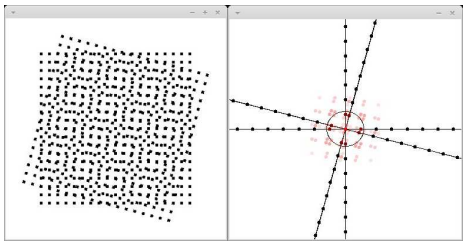


Abbildung 10: Links ist die Überlagerung zweier Punktraster zu sehen, rechts das zugehörige Frequenzspektrum. Die rosa Punkte stehen für die neu hinzugekommenen Impulse.

Bei zu hohen Frequenzen sind die Details zu fein und nicht mehr erkennbar, bei zu niedrigen Frequenzen ist der Betrachter nicht mehr imstande eine periodische Struktur wahrzunehmen. Der Bereich der Frequenzen, die die meisten Menschen als periodische Strukturen erkennen können, kann im Spektrum daher als ein Ring um den Ursprung dargestellt werden, den sogenannten *visibility circle*. Hier übernehme ich die Bezeichnung von Amidror aus [Ami09], auch wenn sie etwas irreführend ist, weil der Kreis ein Loch in der Mitte hat. Liegen die Fundamentalimpulse, das sind die beiden Impulse eines Impulskamms, die am dichtesten am Ursprung liegen, innerhalb des *visibility circle*, so tritt ein Moiré-Muster im Bild auf.

Natürlich ist der *visibility circle* nur ein Konstrukt, der auf Durchschnittswerten beruht. Die Fähigkeit, eine Struktur zu erkennen, ist von vielen Faktoren abhängig, zum Beispiel von den Lichtverhältnissen und dem persönlichen Training des Betrachters. Ich musste in meinem Programm MOIGEN den *visibiltiy circle* mehrmals neu festlegen, bis ich den Eindruck hatte, dass er im Großen und Ganzen meinen Sehfähigkeiten entsprach.

3.1 Binäre Liniengitter

Nach dieser allgemeinen Übersicht will ich mich nun konkreten Bildern und deren Überlagerungen zuwenden.

Liniengitter sind periodische Bilder, die aus parallelen Linien zusammengesetzt sind. Sind diese Linien nur schwarz oder weiß, ohne Grautöne, so ist das Liniengitter binär. Um ein solches Gitter zu untersuchen, muss zunächst seine Reflexionsfunktion $r(x, y)$ bestimmt werden. Offenbar handelt es sich dabei um eine Rechteckwelle, die orthogonal zu ihrer Hauptrichtung konstant ist.

Für den Fall, dass das Gitter nicht gedreht ist, dass also die Linien parallel zur y-Achse verlaufen, ist die Reflexionsfunktion $r_1(x, y)$ symmetrisch zur x-Achse. Ist T_1 die Periode des Gitters und τ_1 die Breite der schwarzen Linien, so gilt

$$r_1(x,y) = \begin{cases} 1 & |x - nT_1| < \tau/2 \\ 0 & |x - nT_1| > \tau/2 \end{cases} \forall n \in \mathbb{Z}.$$

Mithilfe der Fourier-Reihenentwicklung kann man diese Funktion als unendliche

Summe schreiben. Die Berechnung dieser erspare ich mir und dem Leser an dieser Stelle und verweise auf Appendix A in [Ami09]. Es gilt

$$r_1(x,y) = \sum_{-\infty}^{\infty} a_n^{(1)} \cos(2\pi n x/T_1),$$

wobei

$$a_0^{(1)} = \frac{\tau_1}{T_1}$$

und

$$a_n^{(1)} = \frac{1}{n\pi} \sin \frac{\pi n \tau_1}{T_1}.$$

Wird ein gedrehtes Gitter betrachtet, so ist für die Aufstellung der Reflexionsfunktion natürlich der Drehwinkel zu beachten. Auch hier will ich nur das Ergebnis angeben. Die Reflexionsfunktion $r_2(x,y)$ eines um Θ_2 gedrehten Liniengitters mit Periode T_2 und Linienbreite τ_2 ist gegeben durch

$$r_2(x,y) = \sum_{-\infty}^{\infty} a_n^{(2)} \cos(2\pi n[x\cos\Theta_2 + y\sin\Theta_2]/T_2),$$

wobei $a_0^{(2)}$ und $a_n^{(2)}$ entsprechend mit T_2 und τ_2 genau wie oben berechnet werden.

Die Fouriertransformierte $R_1(u,v)$ der Reflexionsfunktion r_1 ist ein symmetrischer Impulskamm auf der u-Achse. Die Intervalle zwischen den Impulsen betragen $1/T_1$ und ihre Amplituden entsprechen den Koeffizienten in der Fourier-Reihenentwicklung $a_n^{(1)}$. Die Fouriertransformierte $R_2(u,v)$ der Reflexionsfunktion r_2 ist ebenfalls ein Impulskamm, der allerdings um den Winkel Θ um den Ursprung gedreht ist. Auch hier haben die Impulse die Amplitude des entsprechenden Koeffizienten in der Reihenentwicklung. Die Frequenzspektren sind in Abb. 11 zu sehen.

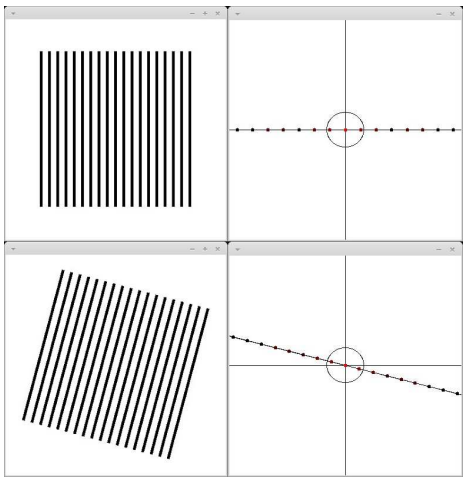


Abbildung 11: Links sind übereinander ein nicht gedrehtes und ein gedrehtes Liniengitter zu sehen, rechts die zugehörigen Frequenzspektren.

Man betrachte nun die Überlagerung zweier Liniengitter $r(x, y) = r_1(x, y)r_2(x, y)$. Das Spektrum des Resultats ist nach dem Konvolutionstheorem durch die Faltung der beiden Spektren $R_1(u, v)$ und $R_2(u, v)$ gegeben. Entsprechend der "move-and-multiply"-Methode ist $R_1 * R_2$ das Spektrum, bei dem über jedem Impuls des Kamms R_1 eine zentrierte Kopie von R_2 liegt. Das ist ein unendliches Impulsbett in der (u, v)-Ebene.

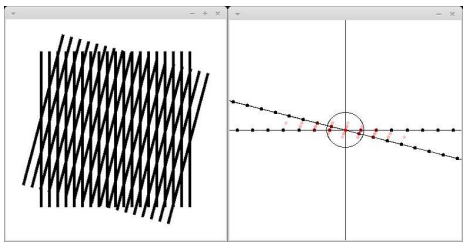


Abbildung 12: Die Überlagerung zweier Liniengitter, im Bild und im Frequenzspektrum.

In Abb. 12 sieht man die neu entstandenen Impulse als rosa Punkte. Sie sind für mögliche Moiré-Effekte verantwortlich. Eigentlich handelt es sich um ein unendliches Impulsbett, doch in meinem Programm MOIGEN wird die Intensität eines Impulses über die Farbsättigung dargestellt, so dass die meisten Impulse zu schwach sind, um erkennbar zu sein. Abb. 13 ist ein Screenshot von einem kleinen Hack, bei dem alle Impulse, unabhängig von ihrer Intensität, zu sehen sind.

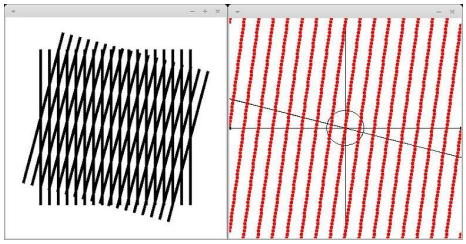


Abbildung 13: Die Überlagerung zweier Liniengitter, im Bild und im Frequenzspektrum ohne Intensitätslimit

Nach "Move-and-Multiply" gilt für die neu hinzugekommenen Impulse:

1. Der Frequenzvektor des (m, n)-ten Impulses im Spektrum der Überlagerung ist die Vektorsumme der Frequenzvektoren des m-ten Impulses des ersten Kamms und des n-ten Impulses des zweiten Kamms. Richtung und Frequenz des Moiré-Musters hängen also nur von dem Linienabstand und der Verdrehung der sich überlagernden Gitter ab. Die *opening ratio* τ/T hat nur Einfluss auf die Koeffizienten $a^{(1)}$ und $a^{(2)}$, spielt also keine Rolle für die geometrische Lage der Impulse im Frequenzspektrum der Überlagerung.

2. Die Amplitude des (m, n)-ten Impulses im Spektrum der Überlagerung ist das Produkt des m-ten Impulses des ersten Kamms mit dem n-ten Impuls des zweiten Kamms, also $a_{m,n} = a_m^{(1)} a_n^{(2)}$. Die Amplitude und die Wellenform des

Moirés hängen damit allein von der *opening ratio* der sich überlagernden Gitter ab. Drehungen der Bilder und Änderungen des Linienabstandes haben keinen Einfluss, denn die Koeffizienten $a^{(1)}$ und $a^{(2)}$ bleiben bei diesen Operationen unverändert.

Das Modell ist erweiterbar auf die Überlagerung von m Liniengittern. Der Frequenzvektor jedes Impulses im Spektrum der Überlagerung ist eine Vektorsumme von m Frequenzvektoren. Dabei ist zu beachten, dass diese auch dem Nullvektor entsprechen können. Die Amplitude jedes Impulses ist das Produkt der Amplituden gemäß seiner vektoriellen Zusammensetzung. Die Auffassung der resultierenden Impulse als Linearkombinationen legt eine Benennung der auftauchenden Moirés über die vektorielle Zusammensetzung ihrer Fundamentalimpulse nahe. Eine einfache Art zu testen, ob ein Gitter an der Entstehung eines Moirés beteiligt ist, ist das Gitter einfach wegzunehmen oder am Computer auszublenden. Bleibt das Moiré erhalten, so war das Gitter nicht notwendig.

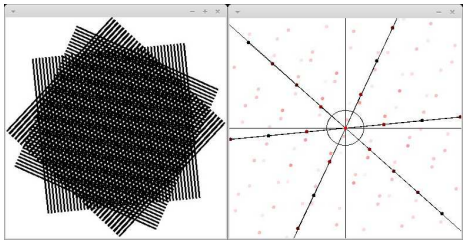


Abbildung 14: Drei überlagerte Liniengitter, alle Gitter sind dabei an der Entstehung des Moiré-Musters beteiligt

Manchmal treten keine Muster in der Überlagerung auf, was verschiedene Gründe haben kann. Im Prinzip gibt es zwei verschiedene Moiré-freie Zustände. Entweder die neu entstandenen Impulse liegen wie in Abb. 15 außerhalb vom *visibility circle*, was bedeutet, dass die Frequenz des entstehenden Moiré-Musters so hoch ist, dass es für den Betrachter unsichtbar bleibt. Oder die Gitter sind gerade so angeordnet, dass die Summe ihrer Frequenzvektoren Null ergibt, wodurch alle starken neuen Impulse im Ursprung liegen wie in Abb. 16. Im ersten Fall ist die Überlagerung stabil Moiré-frei, d.h. auch kleine Änderungen wie z.B. Drehungen ändern nichts an der Tatsache, dass keine Moiré-Muster sichtbar sind. Der zweite Fall ist hingegen Änderungen gegenüber sehr empfindlich. Dreht man eines der Gitter um einen kleinen Winkel, sind im Allgemeinen sofort Moiré-Muster sichtbar.

Jetzt, da der Leser mit den Liniengittern hoffentlich ein wenig vertraut ist, will ich zwei weitere Strukturen vorstellen. Es wird sich herausstellen, dass sowohl Gitternetze als auch Punktraster durch Liniengitter erzeugt werden können, so dass das eben vorgestellte Modell seine Gültigkeit behält.

3.2 Gitternetze

Gitternetze sind nach Liniengittern die wohl einfachste Struktur. Sie entstehen durch Überlagerung von zwei nicht kollinearen Liniengittern. Stehen diese dabei rechtwinklig aufeinander und haben dieselbe Periode, so erhält man ein reguläres

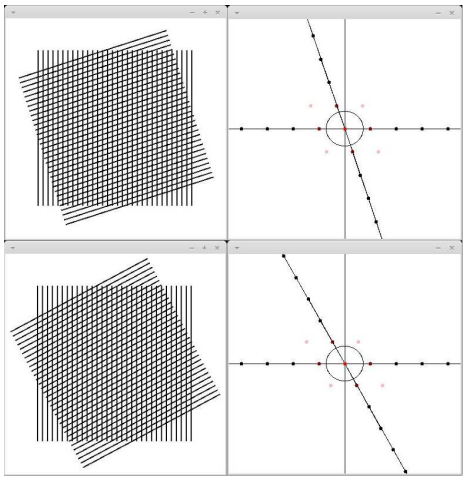


Abbildung 15: Ein stabiler Moiré-freier Zustand. Die neuen Impulse liegen außerhalb vom *visibility circle*. Kleine Änderungen haben keine Auswirkungen.

Gitternetz, auch Quadratgitter genannt. Die Überlagerung von zwei Gitternetzen wie in Abb. 17 entspricht folglich einer Überlagerung von vier Liniengittern, damit gibt es vier Frequenzvektoren im Spektrum, durch deren Kombination Moiré-Muster entstehen können.

3.3 Punktraster

Punktraster sind zweidimensionale Strukturen aus identischen und äquidistanten Punkten, die in parallelen Reihen entlang zweier Achsen angeordnet sind. Stehen diese Achsen senkrecht aufeinander, so ist das Punktraster regulär. Mein Programm MOIGEN bietet zum Experimentieren nur reguläre Punktraster an, auch wenn durch Scherung sehr interessante Effekte erzielt werden können.

Punktraster können als Verallgemeinerung von Quadratgittern angesehen werden, denn Quadratgitter sind reguläre Raster mit weißen, quadratischen Punkten auf schwarzem Grund. Amidror stellt in [Ami09] ein eigenständiges Modell für die Überlagerung von Punktrastern auf und kann damit die Quadratgitter erklären. Ich will andersherum vorgehen und reguläre Punktraster, die quadratische Punkte haben, über Quadratgitter beschreiben.

Ein reguläres Punktraster von schwarzen Quadraten auf weißem Grund kann als Invertierung eines Quadratgitters $r(x, y)$ angesehen werden, also ist die Reflexionsfunktion $s(x, y)$ des Punktrasters gleich $1-r(x, y)$. Für das Frequenzspektrum $S(u, v)$ gilt nach den Rechenregeln aus Kapitel 6 in [Bra78]

$$S(u, v) = \mathcal{F}[s] = \mathcal{F}[1] - \mathcal{F}[-r] = \delta(u, v) - R(u).$$

Das heißt, die Impulse von $S(u, v)$ liegen an gleicher Stelle wie die Impulse von $R(u, v)$, allerdings sind die Amplituden $d_{m,n}$ invertiert:

$$d_{00} = 1 - a_{00},$$

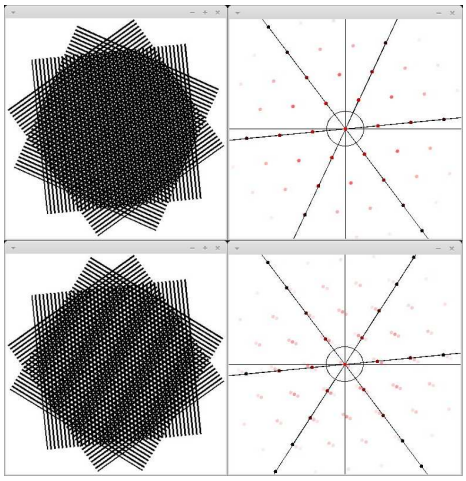


Abbildung 16: Ein instabiler Moiré-freier Zustand. Hier sind die Gitter gerade so angeordnet, dass die Linearkombination der Frequenzvektoren im Ursprung liegt. Eine kleine Änderung sorgt dafür, dass Moiré-Muster auftreten.

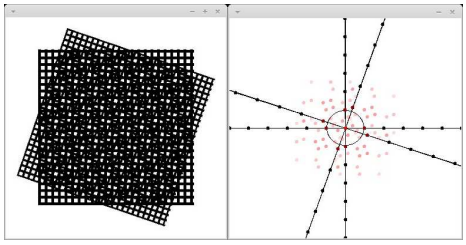


Abbildung 17: Überlagerung zweier Quadratgitter mit unterschiedlicher Strichstärke

$$d_{mn} = -a_{mn} \forall (m,n) \neq (0,0).$$

Zu sehen ist ein Punktgitter in Abb. 18.

3.4 Sampling-Moirés

Sampling-Moirés tauchen auf, wenn ein sich wiederholendes Muster digital abgetastet wird. Beispiele dafür sind z.B. das Fotografieren von feinen Mustern, das Scannen von Zeitungen im Rasterdruck oder die Anzeige von periodischen Funktionen auf dem Bildschirm wie in Abb. 19. In all diesen Beispielen weist das diskrete Signal periodische Anteile, also Wellenmuster auf, die im kontinuierlichen Signal nicht vorhanden waren. Obwohl der Zusammenhang nicht offensichtlich ist, können Sampling-Moirés als Spezialfall des Moiré-Effekts angesehen werden. Dabei ist es eben gerade nicht so, wie in Abb. 20 vermutet wird.

Alle periodischen Anteile einer Funktion liegen im Frequenzspektrum vor. Das gilt ebenfalls für die störenden Wellenmuster im diskreten Signal nach der Abtastung. Deshalb werde ich sein Frequenzspektrum analysieren.

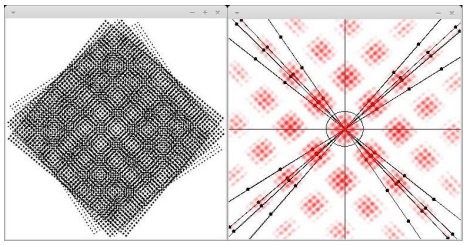


Abbildung 18: Überlagerung dreier Punktgitter

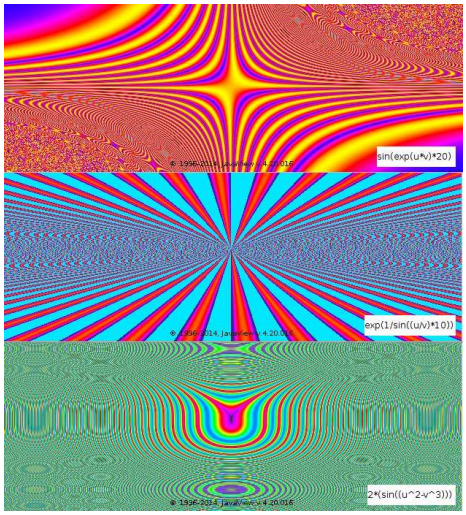


Abbildung 19: Weitere Screenshots aus JavaView mit sichtbaren Sampling-Moirés

Nach dem Abtast-Modell im Abschnitt *Fourier-Theorie und Signalverarbeitung* entspricht die Abtastung einer kontinuierlichen Funktion ihrer Multiplikation mit dem Impulsbett des Abtastgeräts, was in der Moiré-Theorie gleichbedeutend mit einer Überlagerung ist. Das Impulsbett wird also über die Funktion gelegt.

In Abb. 8 entspricht die Funktion, die abgetastet wird, der Reflexionsfunktion $r_1(x, y)$ der Ziegel auf der gemauerten Wand. Die Funktion $r_2(x, y)$ beschreibt das Impulsbett des Abtastgeräts, hier das Pixelgitter auf dem Sensor der Digitalkamera mit Pixelbreite T.

Das Originalbild wird bei der Aufnahme des Fotos an den Stellen $(mT, nT), m, n \in \mathbb{Z}$ abgetastet. Dies entspricht der Multiplikation von $r_1(x, y)$ mit dem Impulsbett $r_2(x, y)$. Das Spektrum $R_(u, v)$ des Fotos ist die Konvolution des Spektrums $R_1(u, v)$ des Bildes mit dem Spektrum $R_2(u, v)$ des Impulsbettes. Das Spektrum $R_2(u, v) = \mathcal{F}[r_1(x, y)]$ ist ein Impulsbett der Schrittbreite $1/T$.

Die Konvolution einer Funktion mit einem Dirac-Impuls an der Stelle (x_0, y_0) erzeugt nach "Move-and-Multiply" oder nach Kapitel 10 in [Bra78] eine Kopie der Funktion, die um die Lage (x_0, y_0) des Dirac-Impulses verschoben ist. Die Konvolution einer Funktion mit einem Impulsbett erzeugt folglich eine Summe von unendlich viele Kopien, wovon jede durch einen Impuls im Impulsbett erzeugt

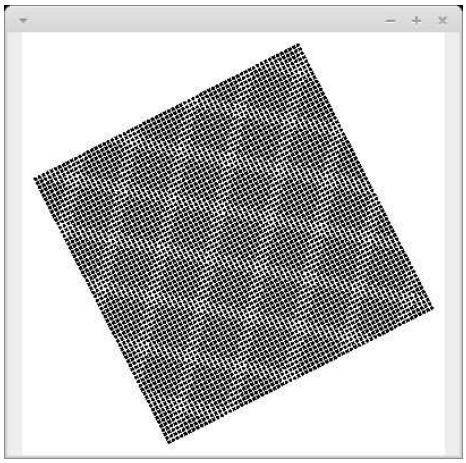


Abbildung 20: Moiré ohne zweites Gitter?

ist und um dessen Lage verschoben ist.

Das Spektrum $R_(u, v)$ besteht demnach aus einer unendlichen Zahl von Kopien von $R_1(u, v)$, jede davon verschoben an die Stelle eines Impulses auf dem Impulsbett $R_2(u, v)$. Ist nun die Schrittweite $1/T$ in $R_2(u, v)$, bekannt als Samplingrate, kleiner als das Doppelte der maximalen Frequenz von $r_1(x, y)$, kommt es zu einer Überlappung der Kopien von $R_1(u, v)$ im Spektrum $R(u, v)$.

Für eine störungsfreie Abtastung muss das Signal $r_1(x, y)$ also mit $\frac{1}{2T}$ bandbegrenzt sein, was in der Digitalfotografie nicht vorkommen wird. Trotzdem treten Moiré-Muster nur bei wenigen Fotos auf. Ist das Spektrum $R_1(u, v)$ glatt und klingt gleichmäßig ab, dann nehmen die Nachbarkopien nur wenig Einfluss aufeinander und der Effekt ist beschränkt auf kleinere Störungen im hohen Frequenzbereich, etwa an scharfen Kanten. Falls es allerdings so ist, dass das Spektrum wegen einer sich wiederholenden Struktur im Bild - in diesem Beispiel das Mauerwerk - starke Ausschläge im hohen Frequenzbereich hat, dann beeinflussen die Nachbarkopien stark die Kopie von $R_1(u, v)$ um den Ursprung. Liegen die starken Ausschläge im hohen Frequenzbereich der Nachbarkopien im *visibility circle*, so sind sie als Störfrequenzen im Bild $r(x, y)$ erkennbar.

3.5 Weitere Strukturen

Vielleicht ist es mir gelungen, mit meinen bisherigen Ausführungen den Leser auf die Überlagerung von Gittern in Form von Kreisen, Parabeln wie in Abb. 21 oder beliebig geschwungenen Linien wie in Abb. 22, nicht-regulären Punktrastern und Punktrastern mit nicht-quadratischen Punkten neugierig zu machen. Ich muss zugeben, dass ich mich mit diesen Strukturen nicht gut genug auskenne, um sie hier vorzustellen. Zu sagen bleibt mir nur, dass das vorgestellte Modell weiterhin zweckmäßig ist:

1. Reflexionsfunktionen zur Beschreibung der Bilder werden aufsgestellt.
2. Die Fouriertransformation liefert die zugehörigen Frequenzspektren.

3. Die Konvolution der Frequenzspektren ergibt das Spektrum der Überlagerung, in dem die Moiré-erzeugenden Impulse untersucht werden können.

Dabei gestaltet es sich oft schon schwierig, die Reflexionsfunktionen zu berechnen. Andererseits verhilft die Vorgehensweise zu einer tiefen Einsicht in die geometrischen Formen von Moiré-Mustern aller Art. Dafür möchte ich auf die hinteren Kapitel in [Ami09] hinweisen. Dort werden neben den Makro- auch die Mikrostrukturen untersucht, wie sie etwa bei der Überlagerung von Punktrastern auftreten.

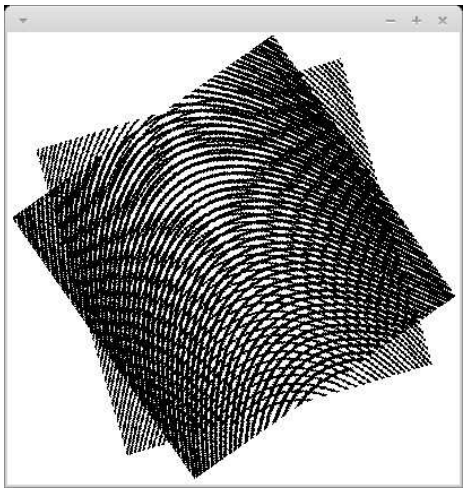


Abbildung 21: Die Überlagerung zweier Parabeln erzeugt eine hyperbolische Form.

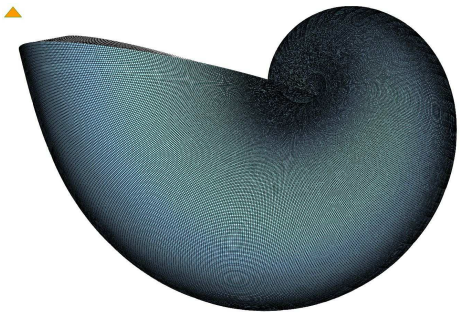


Abbildung 22: Auf diesem Bild aus dem JavaView-Rendering treten hyperbolische und elliptische Moiré-Muster auf. Zu beachten ist, dass kein Sampling-Moiré vorliegt, obwohl es sich um ein computergeneriertes Bild handelt. Verantwortlich ist allein das Netz auf der Oberfläche der Schnecke.

4 Das Programm MOIGEN

Das Programm MOIGEN hat den Zweck, den Anwender mit der Entstehung von Moiré-Mustern im Bildbereich und den entsprechenden Impulsen im Frequenzspektrum vertraut zu machen. Dafür können Liniengitter und Raster beliebiger Breite und Strichstärke übereinandergelegt und gegeneinander verdreht werden.

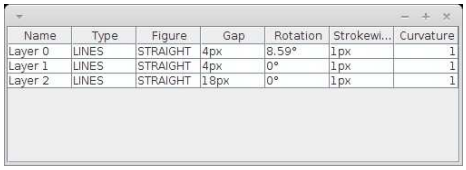
Das Programm besteht aus einem Hauptfenster, zu sehen in Abb. 23, in dem Ebenen hinzugefügt und gelöscht, sowie den eigenen Vorstellungen angepasst werden können. Außerdem besteht die Möglichkeit ein bereits bestehendes Szenario “Preset” zu laden. Dazu kommen drei Anzeigefenster: eine Tabelle (Abb. 24) mit Einträgen zu Strichstärke, Drehwinkel, etc. für jede Ebene, das Bildfenster(Abb. 25), in dem die Überlagerung der Bilder angezeigt wird und das Bildspektrumfenster (Abb. 26), das nur manchmal aktiviert ist.

Der Anwender kann beliebig viele Ebenen übereinanderlegen und sie mit dem Haken links entweder aktivieren oder deaktivieren. Dabei kann er die Typen “LINES” oder “DOTS” und die Formen “STRAIGHT”, “PARABOLA” und “CIRCLE” frei miteinander kombinieren. Für die Form “STRAIGHT” wird das Bildspektrumfenster aktiviert.

Darin kann man das Frequenzspektrum der Ebene beobachten. Der Kreis in der Mitte ist der *visibility circle*. Die durchgezogenen Linien stehen für die Impulskämme der ursprünglichen Bilder, Punkte repräsentieren Impulse, wobei die der ursprünglichen Bilder schwarz, neu Hinzugekommene dagegen rosa sind. Ihre Farbintensität hängt von der Stärke des Impulses ab.



Abbildung 23: Das Hauptfenster. Im Moment sind zwei Ebenen vorhanden.



Name	Type	Figure	Gap	Rotation	Strokewi...	Curvature
Layer 0	LINES	STRAIGHT	4px	8.59°	1px	1
Layer 1	LINES	STRAIGHT	4px	0°	1px	1
Layer 2	LINES	STRAIGHT	18px	0°	1px	1

Abbildung 24: Eine Tabelle zur Übersicht über die gewählten Einstellungen

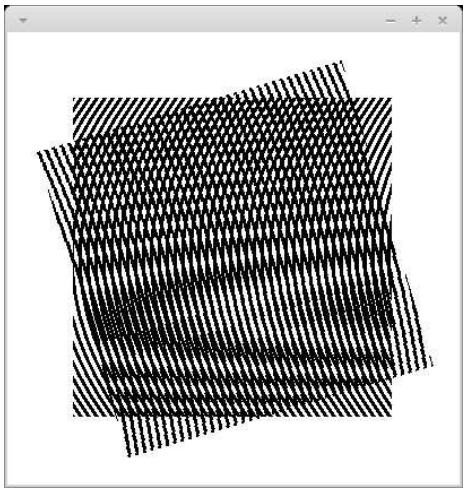


Abbildung 25: Das Bildfenster

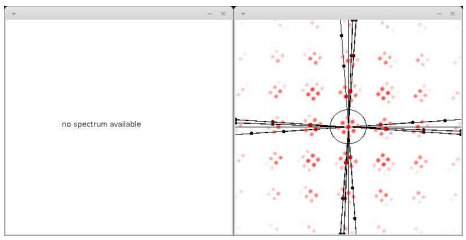


Abbildung 26: Das Bildspektrumfenster, links im deaktvierten Zustand

Literatur

[AJD67] V. J. Parks A. J. Durelli. Moiré fringes as parametric curves. *Experimental Mechanics*, 1967.

[Ami94] Isaac Amidror. A generalized fourier-based method for the analysis of 2d moiré envelope-forms in screen superpositions. *Journal of Modern Optics*, 1994.

[Ami09] Isaac Amidror. *The Theory of the Moiré Phenomenon*. Springer-Verlag, 2009.

[Bra78] Ronald Newbold Bracewell. *The Fourier Transform and Its Applications*. McGraw-Hill, Inc., 1978.

[Bry74] Olof Bryngdahl. Moiré: Formation and interpretation. *Journal of the Optical Society of America*, 1974.

[Kö88] Thomas William Körner. *Fourier Analysis*. Camebridge University Press, 1988.

[Pel12] Colin Pelka. Schloss Lötzen mit Drehbrücke. *de.wikipedia.org/wiki/Moiré-Effekt (6.10.2014)*, 2012.

[Ray74] J. W. Rayleigh. On the manufacture and theory of diffraction-gratings. *Philosophical Magazine and Journal of Science*, 1874.

[Sha49] C. E. Shannon. Communication in the presence of noise. *Proc. Institute of Radio Engineers*, 1949.

```
import javax.swing.*;
import javax.tools.Tool;
import java.awt.*;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.awt.geom.QuadCurve2D;
import java.awt.image.BufferedImage;

import static java.lang.Math.floor;
import static java.lang.Math.max;
import static java.lang.Math.sqrt;

/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 6/28/14
 * Time: 2:20 PM
 */

public class DrawingArea extends JPanel {
    SPanel panel;


    DrawingArea(SPanel panel) {
        this.panel = panel;

    }


    @Override
    public void paint(Graphics g) {

        Dimension areaSize = this.getSize();
        int areaWidth = areaSize.width;

        if (areaWidth != areaSize.height)
            return;


        g.setColor(Color.white);
        g.fillRect(0, 0, areaWidth, areaWidth);
        g.setColor(Color.black);
        float c = 2 * (float) sqrt(2);

        BufferedImage result = new BufferedImage(areaWidth, areaWidth, BufferedImage.TYPE_4BYTE_ABGR);

        Graphics2D rG = (Graphics2D) result.getGraphics();
        for (int i = 0; i < panel.toolboxes.size(); i++) {
            BufferedImage image = new BufferedImage((int) (areaWidth * 2 / c), (int) (areaWidth * 2 / c),
BufferedImage.TYPE_4BYTE_ABGR);

            int imageWidth = (int) (areaWidth * 2 / c);
            Layer layer=panel.toolboxes.get(i).layer;
            if (!layer.isVisible())
                continue;

            int nLines = layer.getNLines();
            int gap = (int) (imageWidth/(nLines*1.0));
            layer.setGap(gap);
            layer.updateStrokewidth();
            int strokewidth = layer.getStrokewidth();



            double phi = layer.getPhi() / 100.0;
            Graphics2D g2 = (Graphics2D) image.getGraphics();
```

```
            g2.setColor(Color.BLACK);

            Layer.Type selectedType = layer.getType();
            switch (selectedType) {
                case LINES:
                    g2.setStroke(new BasicStroke(strokewidth));
                    break;
                case DOTS:
                    float dashPhase=(float) (gap-(Math.ceil((((imageWidth - 1) % gap)/2.0-
strokewidth/2.0))));
                    g2.setStroke(new BasicStroke(strokewidth, BasicStroke.CAP_BUTT,
BasicStroke.JOIN_BEVEL, 0, new float[]{strokewidth, Math.max(0, (gap) - strokewidth)}, dashPhase));
                    break;

            }


            Layer.Figure selectedFigure = layer.getFigure();

            switch (selectedFigure)  {
                case STRAIGHT:
                    for (double x = Math.ceil(((imageWidth - 1) % gap) / 2.0); x <= imageWidth; x+=gap) {

                        Point2D p = new Point2D.Double(x, imageWidth);
                        Point2D q = new Point2D.Double(x, 0);


                        g2.drawLine((int) p.getX(), (int) p.getY(), (int) q.getX(), (int) q.getY());

                    }
                    break;
                case CIRCLE:
                    int linienabstandCirc=max(gap,6);
                    int strokewidthCirc= (int) Math.max(layer.getStrokewidth() * linienabstandCirc /
100.0, 1.0);
                    g2.setStroke(new BasicStroke(strokewidthCirc));
                    for (int r= 0; r < 2*imageWidth; r+=linienabstandCirc) {
                        g2.drawOval( (imageWidth- r) / 2,(imageWidth - r) / 2, r, r);
                    }
                    break;
                case PARABOLA:

                    int parabolaExtension = layer.getCurvature();

                    double extension = (parabolaExtension * 20.0);

                    for (double x = 0; x <= imageWidth * 2; x += gap) {


                        Point2D p = new Point2D.Double(x, imageWidth);
                        Point2D q = new Point2D.Double(x, 0);
                        Point2D u = new Point2D.Double(x - extension, imageWidth/2);



                        QuadCurve2D.Double parabola = new QuadCurve2D.Double(p.getX(), p.getY(), u.getX
(), u.getY(), q.getX(), q.getY());
                        g2.draw(parabola);

                    }
                    break;

            }

            int originalOffset = (int) ((areaWidth - (areaWidth * 2 / c)) / 2);
```

```
            rG.setTransform(AffineTransform.getRotateInstance(phi, (areaWidth / 2), (areaWidth / 2)));

            rG.drawImage(image, originalOffset, originalOffset, null);


        }
        g.drawImage(result, 0, 0, null);
        panel.updateData();
    }

    private void transformPoint(int k, float c, Point2D p) {
        p.setLocation(p.getX() * k * 2 / c, p.getY() * k * 2 / c);
    }


}
```

```java
/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 9/10/14
 * Time: 6:31 PM
 */
public class ImpulseComb {
    Vector2d location;
    double DC;
    int strokewidth;
    int absDistance;


    public ImpulseComb(Vector2d location, double DC, int strokewidth, int absDistance) {
        this.location = location;
        this.DC = DC;
        this.strokewidth = strokewidth;
        this.absDistance = absDistance;
    }

    public Vector2d getLocation() {
        return location;
    }


    public double getDC() {
        return DC;
    }


    public int getStrokewidth() {
        return strokewidth;
    }


    public int getAbsDistance() {
        return absDistance;
    }

}
```

```java
/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 8/25/14
 * Time: 7:30 PM
 */
public class Layer {

    public enum Type {
        LINES,
        DOTS

    }

    public enum Figure {
        STRAIGHT,
        PARABOLA,
        CIRCLE
    }

    private Figure figure;
    private Type type;
    private int id;
    private int nLines;
    private int gap;
    private int phi;
    private int openingRatio;          // in percent

    private int strokewidth;
    private int curvature;
    private boolean visible;

    public Layer(int id) {
        this.id = id;
        this.figure = Figure.STRAIGHT;
        this.type = Type.LINES;
        this.nLines = 15;
        this.phi = 0;
        this.openingRatio=10;
        this.strokewidth = 1;
        this.curvature = 1;
        this.visible = true;
    }

    public int getGap() {
        return gap;
    }

    public void setGap(int gap) {
        this.gap = gap;
    }

    public int getStrokewidth() {
        return strokewidth;
    }

    public void setStrokewidth(int strokewidth) {
        this.strokewidth = strokewidth;
    }

    public int getOpeningRatio() {
        return openingRatio;
    }

    public void setOpeningRatio(int openingRatio) {
        this.openingRatio = openingRatio;
    }
```

```
    public void updateStrokewidth() {
        setStrokewidth((Math.max(openingRatio*gap/100,1)));


    }

    public Figure getFigure() {
        return figure;
    }

    public void setFigure(Figure figure) {
        this.figure = figure;
    }

    public Type getType() {
        return type;
    }

    public void setType(Type type) {
        this.type = type;
    }

    public int getNLines() {
        return nLines;
    }

    public void setNLines(int nLines) {
        this.nLines = nLines;
    }

    public int getPhi() {
        return phi;
    }

    public void setPhi(int phi) {
        this.phi = phi;
    }

    public int getCurvature() {
        return curvature;
    }

    public void setCurvature(int curvature) {
        this.curvature = curvature;
    }

    public boolean isVisible() {
        return visible;
    }

    public void setVisible(boolean visible) {
        this.visible = visible;
    }



    public int getId() {
        return id;
    }

}
```

```
import javax.swing.*;
import javax.swing.table.AbstractTableModel;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.TableCellRenderer;
import java.awt.*;
import java.text.NumberFormat;
import java.util.ArrayList;

/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 8/25/14
 * Time: 7:25 PM
 */
public class LayerPropertyTable extends JTable {

    private final PropertyTableModel dataModel;

    public LayerPropertyTable(SPanel sPanel) {
        this.sPanel = sPanel;
        createColumns();
        dataModel = new PropertyTableModel();
        setModel(dataModel);

    }

    public void update() {
        dataModel.update();
        repaint();
    }

    public void createColumns() {
        columns = new ArrayList<Column>();
        columns.add(new Column("Name",String.class) {
            @Override
            Object getValue(Layer layer) {
                return "Layer "+layer.getId();
            }
        });
        columns.add(new Column("Type",String.class) {
            @Override
            Object getValue(Layer layer) {
                return layer.getType();
            }

        });
        columns.add(new Column("Figure",String.class) {
            @Override
            Object getValue(Layer layer) {
                return layer.getFigure();
            }

        });


        columns.add(new Column("Gap",String.class) {
            @Override
            Object getValue(Layer layer) {
                return layer.getGap()+"px";
            }
        });
        columns.add(new Column("Rotation",String.class) {



            @Override
            Object getValue(Layer layer) {
```

```
                double degree=layer.getPhi()*1.8/Math.PI;
                NumberFormat format= NumberFormat.getNumberInstance();
                format.setMaximumFractionDigits(2);
                return(format.format(degree)+"°");
            }

        });
        columns.add(new Column("Strokewidth",String.class) {
            @Override
            Object getValue(Layer layer) {
                return layer.getStrokewidth()+"px";
            }

        });
        columns.add(new Column("Curvature",Integer.class) {
            @Override
            Object getValue(Layer layer) {
                return layer.getCurvature();
            }

        });



    }

    private ArrayList<Column> columns;
    private SPanel sPanel;

    public class PropertyTableModel extends AbstractTableModel {

        public void update() {
            fireTableDataChanged();
        }

        @Override
        public int getRowCount() {
            return sPanel.toolboxes.size();
        }

        @Override
        public String getColumnName(int column) {
            return columns.get(column).name;
        }

        @Override
        public Class<?> getColumnClass(int columnIndex) {
            return columns.get(columnIndex).type;
        }

        @Override
        public int getColumnCount() {
            return columns.size();
        }

        @Override
        public Object getValueAt(int rowIndex, int columnIndex) {
            return columns.get(columnIndex).getValue(sPanel.toolboxes.get(rowIndex).layer);
        }

    }


    public abstract static class Column {

        private String name;
```

```
        private Class type;

        abstract Object getValue(Layer layer);

        protected Column(String name, Class type) {
            this.name = name;
            this.type = type;

        }


    }


}
```

```java
import com.sun.java.swing.plaf.gtk.GTKLookAndFeel;
import com.sun.java.swing.plaf.windows.WindowsClassicLookAndFeel;
import com.sun.java.swing.plaf.windows.WindowsLookAndFeel;

import javax.swing.*;
import javax.swing.plaf.multi.MultiLookAndFeel;
import javax.swing.plaf.nimbus.NimbusLookAndFeel;
import javax.swing.plaf.synth.SynthLookAndFeel;

import java.awt.*;

import static javax.swing.WindowConstants.EXIT_ON_CLOSE;

/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 6/28/14
 * Time: 2:19 PM
 */
public class MainClass {

    public static void main(String[] args) throws UnsupportedLookAndFeelException {
        try {
            UIManager.setLookAndFeel(new WindowsClassicLookAndFeel());
        } catch (UnsupportedLookAndFeelException e) {
            e.printStackTrace();
        }


        JFrame frame=new JFrame();
        SPanel sPanel=new SPanel(frame);
        frame.add(sPanel.surrounder2);
        frame.pack();
        frame.setVisible(true);
        frame.setDefaultCloseOperation(EXIT_ON_CLOSE);



        JFrame drawingAreaFrame = new JFrame();
        drawingAreaFrame.add(sPanel.surrounder);


        drawingAreaFrame.pack();
        drawingAreaFrame.setVisible(true);

        JFrame tableFrame = new JFrame();
        LayerPropertyTable table = new LayerPropertyTable(sPanel);
        JScrollPane scrollpane = new JScrollPane(table);
        scrollpane.setPreferredSize(new Dimension(500,150));

        tableFrame.add(scrollpane);
        tableFrame.pack();
        tableFrame.setVisible(true);
        sPanel.setTable(table);

        JFrame spectrumFrame=new JFrame();
        SpectrumPanel spectrumPanel=new SpectrumPanel(sPanel);
        spectrumFrame.getContentPane().add(spectrumPanel);
        spectrumFrame.setSize(400,400);
        spectrumFrame.setLocation(400,400);
        spectrumFrame.setVisible(true);
        spectrumFrame.setResizable(false);
        sPanel.setSpectrumPanel(spectrumPanel);

    }
}
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ComponentAdapter;
import java.awt.event.ComponentEvent;
import java.util.ArrayList;

/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 6/28/14
 * Time: 2:19 PM
 */
public class SPanel extends JPanel {
    JFrame frame;
    DrawingArea drawingArea;
    JComboBox presetBox;
    JPanel toolpanel;
    JPanel surrounder;
    JPanel surrounder2;


    int nLayers;
    ArrayList<Toolbox> toolboxes;

    private LayerPropertyTable table;
    private SpectrumPanel spectrumPanel;

    public enum Presets {
        PRESET_1,
        PRESET_2,
        PRESET_3,
        PRESET_4

    }


    public void setTable(LayerPropertyTable table) {
        this.table = table;
    }

    SPanel(JFrame frame) {
        super();
        this.frame = frame;

        this.setLayout(new BorderLayout());
        toolpanel = new JPanel();

        BoxLayout boxLayout = new BoxLayout(toolpanel, BoxLayout.PAGE_AXIS);

        toolpanel.setLayout(boxLayout);
        nLayers = 0;
        toolboxes = new ArrayList<Toolbox>();

        JButton newLayerButton = new JButton("New Layer");
        newLayerButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                createNewLayer();
            }
        });

        JPanel newLayerButtonAndpresetBoxPanel = new JPanel();
        newLayerButtonAndpresetBoxPanel.add(newLayerButton);

        JButton preset1Button = new JButton("Preset 1");
```

```
        preset1Button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                removeAllLayers();
                createNewLayer();
                createNewLayer();
                createNewLayer();
                toolboxes.get(0).densitySlider.setValue(40);
                toolboxes.get(1).densitySlider.setValue(40);
                toolboxes.get(2).densitySlider.setValue(40);
                toolboxes.get(0).strokeslider.setValue(50);
                toolboxes.get(1).strokeslider.setValue(50);
                toolboxes.get(2).strokeslider.setValue(50);
                toolboxes.get(0).phislider.setValue(304);
                toolboxes.get(1).phislider.setValue(95);
                toolboxes.get(2).phislider.setValue(199);

            }
        });

        JButton preset2Button = new JButton("Preset 2");
        preset2Button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                removeAllLayers();
                createNewLayer();
                createNewLayer();
                toolboxes.get(1).figureBox.setSelectedItem(Layer.Figure.CIRCLE);
                toolboxes.get(0).densitySlider.setValue(38);
                toolboxes.get(1).densitySlider.setValue(23);
                toolboxes.get(0).phislider.setValue(40);
            }
        });

        JButton preset3Button = new JButton("Preset 3");
        preset3Button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                removeAllLayers();
                for (int i = 0; i < 4; i++) {
                    createNewLayer();
                    toolboxes.get(i).typeBox.setSelectedItem(Layer.Type.DOTS);
                    ;

                }
                toolboxes.get(0).densitySlider.setValue(47);
                toolboxes.get(1).densitySlider.setValue(45);
                toolboxes.get(2).densitySlider.setValue(44);
                toolboxes.get(3).densitySlider.setValue(47);

                toolboxes.get(0).strokeslider.setValue(47);
                toolboxes.get(1).strokeslider.setValue(47);
                toolboxes.get(2).strokeslider.setValue(50);
                toolboxes.get(3).strokeslider.setValue(55);

                toolboxes.get(0).phislider.setValue(99);
                toolboxes.get(1).phislider.setValue(80);
                toolboxes.get(2).phislider.setValue(84);
                toolboxes.get(3).phislider.setValue(69);

            }
        });
        JButton preset4Button = new JButton("Preset 4");
        preset4Button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                removeAllLayers();
```

```
                createNewLayer();
                createNewLayer();
                toolboxes.get(0).figureBox.setSelectedItem(Layer.Figure.PARABOLA);
                toolboxes.get(1).figureBox.setSelectedItem(Layer.Figure.PARABOLA);
                toolboxes.get(0).densitySlider.setValue(40);
                toolboxes.get(1).densitySlider.setValue(40);
                toolboxes.get(0).strokeslider.setValue(51);
                toolboxes.get(1).strokeslider.setValue(50);
                toolboxes.get(0).parabolaSlider.setValue(5);
                toolboxes.get(1).parabolaSlider.setValue(8);
                toolboxes.get(1).phislider.setValue(285);

            }
        });


        newLayerButtonAndpresetBoxPanel.add(preset1Button);
        newLayerButtonAndpresetBoxPanel.add(preset2Button);
        newLayerButtonAndpresetBoxPanel.add(preset3Button);
        newLayerButtonAndpresetBoxPanel.add(preset4Button);

        toolpanel.add(newLayerButtonAndpresetBoxPanel);


        surrounder2 = new JPanel();
        surrounder2.add(toolpanel);

        drawingArea = new DrawingArea(this);
        surrounder = new JPanel();

        surrounder.setLayout(null);
        surrounder.setPreferredSize(new Dimension(400, 400));
        surrounder.add(drawingArea);

        surrounder.addComponentListener(new ComponentAdapter() {
            @Override
            public void componentResized(ComponentEvent e) {

                Object source = e.getSource();
                JPanel s = (JPanel) source;

                Dimension size = s.getSize();


                int min = Math.min(size.width, size.height);
                drawingArea.setSize(new Dimension(min, min));
                int x = (size.width - min) / 2;
                int y = (size.height - min) / 2;

                drawingArea.setLocation(x, y);


            }
        });


    }

    void createNewLayer() {

        final Toolbox box = new Toolbox(this, new Layer(nLayers));
        toolboxes.add(box);
        nLayers++;
        toolpanel.add(box);
        toolpanel.setVisible(false);
        toolpanel.setVisible(true);
        update();
```

```
        this.frame.pack();

    }

    public void update() {

        drawingArea.repaint();
        updateData();
    }


    public void removeLayer(Layer layer) {
        for (int i = 0; i < toolboxes.size(); i++) {
            if (toolboxes.get(i).layer == layer) {
                toolboxes.remove(i);
                toolpanel.remove(i + 1);
            }
        }
        toolpanel.invalidate();
        toolpanel.revalidate();
        update();
        frame.pack();

    }

    public void removeAllLayers() {
        while (!toolboxes.isEmpty()) {

            removeLayer(toolboxes.get(0).layer);
        }
    }

    public void updateData() {
        if (table != null) {
            table.update();

        }
        if (spectrumPanel != null) {
            spectrumPanel.repaint();
        }
    }

    public void setSpectrumPanel(SpectrumPanel spectrumPanel) {
        this.spectrumPanel = spectrumPanel;
    }
}
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ComponentAdapter;
import java.awt.event.ComponentEvent;
import java.util.ArrayList;

/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 8/26/14
 * Time: 11:34 AM
 */
public class SpectrumPanel extends JPanel {
    SPanel sPanel;

    private static final double xMin = -0.5;
    private static final double xMax = 0.5;
    private static final double yMin = -0.5;
    private static final double yMax = 0.5;
    double maxRange = Math.sqrt(2) * Math.max(Math.max(Math.abs(xMin), xMax), Math.max(Math.abs(yMin),
yMax));

    double threshold = 0.02;
    double visibilityRadius = 0.08;


    private int xPixelPerUnit;
    private int yPixelPerUnit;


    public SpectrumPanel(SPanel sPanel) {
        this.sPanel = sPanel;
        this.setBackground(Color.white);
        addComponentListener(new ComponentAdapter() {
            @Override
            public void componentResized(ComponentEvent e) {
                updateResolution();
            }
        });
    }

    private void updateResolution() {
        xPixelPerUnit = (int) (getWidth() / (xMax - xMin));
        yPixelPerUnit = (int) (getHeight() / (yMax - yMin));
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g) {

        super.paintComponent(g);
        for (Toolbox toolbox : sPanel.toolboxes) {

            if (toolbox.layer.getFigure().equals(Layer.Figure.CIRCLE) || toolbox.layer.getFigure().equals
(Layer.Figure.PARABOLA)) {
                g.drawString("no spectrum available", getWidth() / 4, getHeight() / 2);

                return;
            }
        }

        g.setColor(Color.BLACK);
        g.drawLine(getWidth() / 2, 0, getWidth() / 2, getHeight());
        g.drawLine(0, getHeight() / 2, getWidth(), getHeight() / 2);
        g.drawLine(getWidth() / 2, getHeight() - (int) ((1 - yMin) * yPixelPerUnit), getWidth() / 2 - 5,
getHeight() - (int) ((1 - yMin) * yPixelPerUnit));
```

```
        g.drawOval((int) ((-visibilityRadius - xMin) * xPixelPerUnit), (int) (getHeight() -
(visibilityRadius - yMin) * getHeight() / (yMax - yMin)), (int) (2 * visibilityRadius * xPixelPerUnit),
(int) (2 * visibilityRadius * yPixelPerUnit));


        ArrayList<ImpulseComb> allCombs = new ArrayList<ImpulseComb>();
        double maxDC = 0;


        if (sPanel.toolboxes.size() == 0)
            return;


        for (Toolbox toolbox : sPanel.toolboxes) {
            Layer layer = toolbox.layer;

            if (layer.isVisible() == false) {
                continue;
            }


            int absDistance = layer.getGap();
            int strokewidth = layer.getStrokewidth();
            double theta = layer.getPhi() / 100.0 + Math.PI / 2;

            Vector2d v = new Vector2d(Math.sin(theta), Math.cos(theta));

            v = v.multiply(1.0 / absDistance);
            double dc = strokewidth / (absDistance * 1.0);
            maxDC = Math.max(dc, maxDC);
            allCombs.add(new ImpulseComb(v, dc, strokewidth, absDistance));


            if (layer.getType() == Layer.Type.DOTS) {
                allCombs.add(new ImpulseComb(v.turn(Math.PI / 2.0), dc, strokewidth, absDistance));

            }

        }


        for (ImpulseComb comb : allCombs) {
            Vector2d v = comb.getLocation();
            int i = 0;
            while (v.multiply(i).getLength() < maxRange) {
                i++;
                Vector2d multiply = v.multiply(i);
                drawVector(g, multiply, false);
                drawVector(g, multiply.invert(), false);

            }
        }


        int m = (int) (Math.pow(maxDC, allCombs.size()) / (Math.PI * threshold)) + 1;


        g.setColor(Color.RED);

        for (int i = 0; i < (Math.pow(2 * m + 1, allCombs.size())); i++) {
            Vector2d res = new Vector2d(0, 0);
            double impulse = 1;
            int x = i;

            for (ImpulseComb comb : allCombs) {

                Vector2d v = comb.getLocation();
```

```
                int k = x % (2 * m + 1) - m;
                if (k != 0) {
                    impulse = Math.abs(impulse * 1 / (k * Math.PI) * Math.sin(Math.PI * k *
comb.getStrokewidth() / (comb.getAbsDistance() * 1.0)));
                }


                x = x / (2 * m + 1);
                res = res.add(v.multiply(k));

            }

            if (res.isBetween(xMin, xMax, yMin, yMax)) {
                //g.setColor(Color.red);
                g.setColor(new Color((float) 1, (float) 0, (float) 0, (float) (Math.pow(impulse, 0.4))));
                drawVector(g, res, true);
            }


        }


    }

    private void drawVector(Graphics g, Vector2d v, boolean onlyEndPoint) {
        g.fillOval((int) ((v.x - xMin) * xPixelPerUnit) - 3, (int) (getHeight() - (v.y - yMin) * getHeight
() / (yMax - yMin)) - 3, 6, 6);
        if (!onlyEndPoint)
            g.drawLine(getWidth() / 2, getHeight() / 2, (int) ((v.x - xMin) * xPixelPerUnit), (int)
(getHeight() - (v.y - yMin) * getHeight() / (yMax - yMin)));
    }

}
```

```
import javax.swing.*;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;

/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 7/28/14
 * Time: 11:07 AM
 */
public class Toolbox extends JPanel {
    Layer layer;
    SPanel sPanel;


    Toolbox(SPanel spanel, Layer layer) {
        this.sPanel = spanel;
        this.layer = layer;
        createUI();
    }

    private void createUI() {
        visibleBox = new JCheckBox("Layer " + layer.getId(), true);
        visibleBox.addItemListener(new ItemListener() {
            @Override
            public void itemStateChanged(ItemEvent e) {
                layer.setVisible(visibleBox.isSelected());
                sPanel.update();

            }
        });
        add(visibleBox);


        typeBox = new JComboBox(Layer.Type.values());
        typeBox.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                layer.setType((Layer.Type) typeBox.getSelectedItem());
                sPanel.update();
            }
        });

        add(typeBox);


        figureBox = new JComboBox(Layer.Figure.values());
        figureBox.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                parabolaSlider.setEnabled(figureBox.getSelectedItem() == Layer.Figure.PARABOLA);
                layer.setFigure((Layer.Figure) figureBox.getSelectedItem());
                sPanel.update();
            }
        });
        add(figureBox);

        densitySlider = new JSlider(1, 100, 15);

        densitySlider.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent e) {
```

```
                if (!densitySlider.getValueIsAdjusting()) {
                    layer.setNLines(densitySlider.getValue());

                    sPanel.update();
                }
            }
        });
        ;
        densitySlider.setPreferredSize(new Dimension(100, (int) densitySlider.getPreferredSize().getHeight
()));
        add(new JLabel("Density:"));
        add(densitySlider);

        phislider = new JSlider(0, (int) (100 * Math.PI), 0);
        phislider.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent e) {

                if (!phislider.getValueIsAdjusting()) {
                    layer.setPhi(phislider.getValue());

                    sPanel.update();
                }
            }
        });
        ;
        add(new JLabel("Rotation:"));
        phislider.setPreferredSize(new Dimension(100, (int) phislider.getPreferredSize().getHeight()));
        add(phislider);

        strokeslider = new JSlider(0, 100, 10);
        strokeslider.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent e) {

                if (!strokeslider.getValueIsAdjusting()) {
                    layer.setOpeningRatio(strokeslider.getValue());

                    sPanel.update();
                }
            }
        });

        add(new JLabel("Opening Ratio:"));
        strokeslider.setPreferredSize(new Dimension(100, strokeslider.getPreferredSize().height));
        add(strokeslider);

        parabolaSlider = new JSlider(1, 15);
        parabolaSlider.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent e) {

                if (!parabolaSlider.getValueIsAdjusting()) {
                    layer.setCurvature(parabolaSlider.getValue());

                    sPanel.update();
                }
            }
        });
        add(new JLabel("Curvature:"));
        parabolaSlider.setPreferredSize(new Dimension(100, parabolaSlider.getPreferredSize().height));
        add(parabolaSlider);

        parabolaSlider.setEnabled(false);

        JButton removeButton = new JButton("Remove");
```

```
        removeButton.setActionCommand("remove" + layer.getId());
        removeButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                sPanel.removeLayer(layer);
            }
        });
        removebutton = removeButton;
        add(removeButton);
    }


    JComboBox figureBox;
    JComboBox typeBox;
    JSlider phislider;
    JSlider densitySlider;
    JSlider strokeslider;
    JSlider parabolaSlider;
    JCheckBox visibleBox;
    JButton removebutton;


}
```

```java
/**
 * Created with IntelliJ IDEA.
 * User: ullika
 * Date: 8/26/14
 * Time: 12:14 PM
 */
public class Vector2d {
    public double x;
    public double y;

    public Vector2d(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public Vector2d add(Vector2d v) {
        return new Vector2d(x + v.x, y + v.y);
    }

    public Vector2d multiply(double k) {
        return new Vector2d(k * x, k * y);
    }

    public Vector2d invert() {
        return this.multiply(-1);
    }


    public Vector2d turn(double phi) {
        return new Vector2d(x * Math.cos(phi) - y * Math.sin(phi), x * Math.sin(phi) + y * Math.cos(phi));

    }

    public double getLength() {
        return Math.sqrt(x * x + y * y);
    }

    public boolean isBetween(double xMin, double xMax, double yMin, double yMax) {
        return x > xMin && y > yMin && x < xMax && y < yMax;
    }

    @Override
    public String toString() {
        return "[" + x + "|" + y + "]";
    }
}
```

Erklärung Hiermit erkläre ich, dass ich die vorliegende Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Hausarbeit, die anderen Quellen im Wortlaut oder dem Sinn nach entnommen wurden, sind durch Angaben der Herkunft kenntlich gemacht. Dies gilt auch für Zeichnungen, Skizzen, bildliche Darstellungen sowie für Quellen aus dem Internet.